

ТЕМА 5. СТРУКТУРИ ЗА КОНТРОЛ

СТРУКТУРНИ ОПЕРАТОРИ В ЕП

Структурните оператори (управляващи структури, оператори-контроли, *control statements*) — такива оператори, които в своята структура съдържат други оператори, които от своя страна се наричат **вложени оператори**



По този начин с помощта на съставните оператори могат да се конструират йерархии от оператори, съдържащи се един в друг

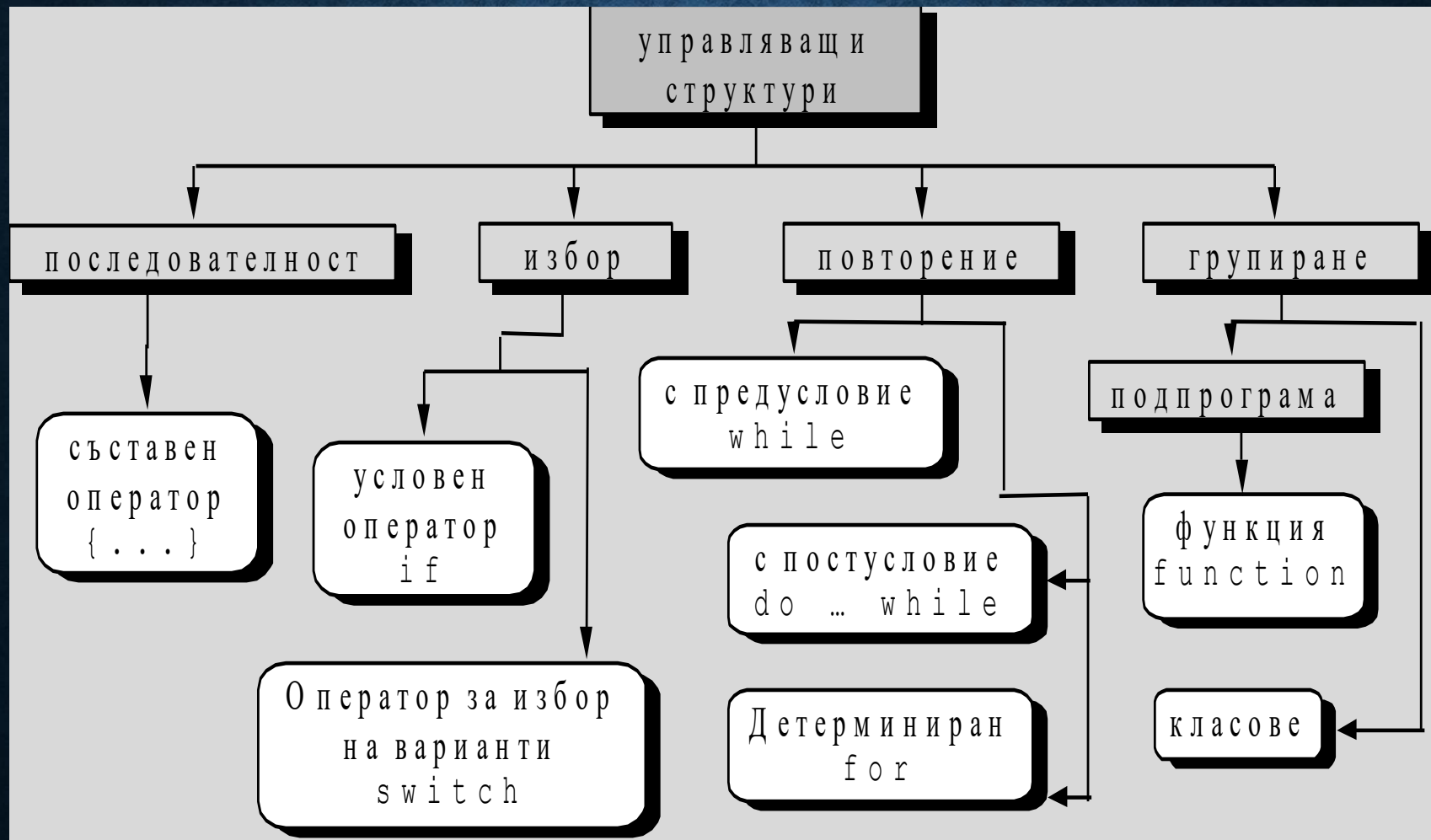


Всеки един от операторите в йерархията е разположен на определено **ниво на вложеност** - от най-външно, до най-вътрешно (! в общия случай се изпълняват всички оператори на вътрешното ниво и тогава се преминава към тези на по-горно ниво)

Видове структурни оператори:

- **Последователност**
- **Избор** при изпълнението на една или няколко стъпки
- **Повторение** при изпълнението на една или няколко стъпки
- **Групиране** — служат за групиране на множество от оператори или данни (ще бъдат разгледани по-късно)

СЪСТАВНИ ОПЕРАТОРИ В C++



ОПЕРАТОРИ ЗА ПОСЛЕДОВАТЕЛНОСТ: СЪСТАВЕН ОПЕРАТОР

Същност

Съставният оператор обединява линейно подредено множество от оператори, които задават определена цел при обработката на данните и се изпълняват последователно по реда на написването им

Синтаксис

Съставният оператор се нарича още **блок**

- **Синтаксис:**

```
{  
    <оператор1> незад;  
    <оператор2> незад;  
    ... <операторn> незад;  
}
```

- **Семантика:**

Изпълняват се последователно операторите между { и } по реда на срещането им

- **Примери:**

```
{  
    D=sqrt (D) ;  X1=(-B+D) / (2*A) ;  X2=(-B-D) / (2*A) ;  
}
```

- Тъй като в C++ декларациите се считат за оператори, то списъкът с оператори между { и } може да съдържа декларации. В резултат на това всяко име декларирано в даден блок е валидно (видимо) само в неговите рамки (е локално за блока, има **локална област на действие**)

УПРАВЛЯВАЩИ СТРУКТУРИ ЗА ИЗБОР

Управляващите структури за избор определят дали даден оператор (или последователност от оператори) трябва да бъде изпълнен или не, обикновено чрез изследване стойността на някакъв израз

- **Приложение:**

В случаи, при които това коя част от обработката ще се извърши зависи от конкретните данни (напр. ако е необходимо да се направи отстъпка в цената на стоки в зависимост от срок на годност или категория.) Дайте други примери. В такъв случай в зависимост от данните се извършват различни изчисления.

- **Видове оператори за избор в C++:**

- **условен оператор** (оператор `if`) – изборът се прави като се изследва стойността на логически израз
- **оператори за избор на варианти** (оператор `switch...case`) – изборът е въз основа на стойността на дискретен израз

УСЛОВЕН ОПЕРАТОР IF

Същност

Операторът дава възможност за условно изпълнение на някакво действие или избор между две независими действия според стойността на логически израз

Т.е. даден/и оператор/и (в общия смисъл) ще бъдат изпълнени само при определени условия

- **Синтаксис:**

```
(1)      if (<израз>)
           <оператор1>;
           else
           <оператор2>;
```

```
(2)      if (<израз>)
           <оператор1>;
```

- **Семантика:** Ако стойността на <израз> е различна от нула, <оператор₁> се изпълнява. Ако стойността на <израз> е нула:

- при (1) се изпълнява <оператор₂> (след `else`)
- при (2) просто се продължава изпълнението на програмата със следващия оператор

УСЛОВЕН ОПЕРАТОР – ПРИМЕРИ

- (1) **кратка форма**

```
if((exam mark > 4) && (assignment_mark > 4))  
    good_student++;
```
- (2) **със съставен оператор, задължително е използването на блок {...}**

```
if((exam_mark > 4) && (assignment_mark > 4))  
{  
    cout << "*";  
    good_student++;  
}
```
- (3) **пълна форма**

```
if(gender tag == 'F')  
    females++;  
else  
    males++;
```
- (4) **вложени условни оператори**

```
if(mark>=3 && mark<=4) {  
    S_Count++;  
    cout << "Good job!" << endl;  
}  
else  
    if(mark<=5)  
        M_Count++;  
    else  
        if(mark<=6)  
        {  
            O_Count++;  
            cout << "Excellent!" << endl;  
        }  
}
```

УСЛОВЕН ОПЕРАТОР – ОСОБЕНОСТИ

- синтаксисът изисква един единствен оператор, както след `if`, така и след `else`
- операторът след `if` завършва с `;` само ако не е съставен (виж примерите)
- при вложени оператори `if` клаузата `else` се свързва винаги с най-близкия предхождащ `if` от същото ниво на вложеност (ако искаме да избегнем действието на това правило е необходимо да използваме допълнителен съставен оператор):

```
if(mark<3) {  
    if((mark == 2) )  
        cout << "Bad situation!" << endl;  
}  
else  
    if(mark>2.5) ...
```

- в `if` не може да се използва оператора за последователно остойносттаване (`“ , ”`)

ПРИЛОЖЕНИЕ

- Широко – примери?
- На практика няма програма, която да не съдържа множество условни оператори
- Валидация на входни данни (напр. дали стойност е в определени граници), при деление проверка за стойност 0, др.

```
if (x >= 20 && x <= 40)
```

- Особено внимание с използването на ;

```
int x = 5, y = 10;
```

```
cout << "x is " << x << " and y is " << y << endl;
```

```
if (x > y); // грешка с използването на ;
```

```
cout << "x is greater than y \n"; //извеждане ще бъде правено винаги
```

Резултат

```
x is 5 and y is 10
```

```
x is greater than y
```

УСЛОВЕН ОПЕРАТОР-ИЗРАЗ

Условният оператор-израз (*Conditional Operator*) се използва за изчисляване на точно един от два възможни изрази.

Той е три-аргументен. Може да се използва като алтернатива на if / else за кратки изрази.

- **Синтаксис:**

`<израз> ? < израз 1> : < израз 2>;`

, където <израз> е от прост тип

- **Семантика:**

Ако стойността на <израз> е различна от нула, < израз ₁> се остойностява. Ако стойността на <израз> е нула, то се остойностява < израз ₂>. Така получената стойност е резултатът от изпълнението на условния оператор-израз

- **Примери:**

```
salary = (Employ_Status == 'G') ? 600 : 450;
```

```
maximum=(temperature > maximum) ? temperature :maximum;
```

```
cout << ((gender_tag == 'f') ? "Female : " : "Male : ");
```

ОПЕРАТОР ЗА ИЗБОР НА ВАРИАНТИ В C++

Същност

Този оператор определя изпълнението на едно от няколко възможни действия (варианта). В езика C++ тези действия не са независими, т.е. могат да бъдат изпълнени няколко от тях или нито едно

- **Синтаксис:**

```
switch( <израз> )  
{  
    case <константен израз> : незад ... <оператор> незад ; break; незад  
    default: <оператор> незад  
}
```

,където:

- <израз> трябва да е дискретен (да може да се остойносттава като цяло число), или променлива от целочислен тип, вкл. char
- всяка case клауза асоциира (съпоставя) цяло число със съответния вариант от възможни действия
- всеки <константен израз> е от типа на <израз>

- **Семантика:** Изчислява се стойността на <израз>, след което изпълнението продължава от оператора на този вариант, за който <константен израз> е равен на изчислената стойност. В случай, че тя не се съответства на никой вариант, то се изпълнява операторът след default (ако има такъв). Изпълнението на оператора продължава до неговия край (}) или до срещането на оператора **break**

ОПЕРАТОР ЗА ИЗБОР НА ВАРИАНТИ В ЕЗИКА C++ – ПРИМЕРИ

(1)

```
switch( c )
{
    case 'A':
    case 'a':
        lettera++; break;
    default :
        other++;
}
```

(2)

```
switch( i )
{
    case -1:
        n++;
        break;
    case 0 :
        z++;
        break;
    case 1 :
        p++;
        break;
}
```

Приложение:

- Когато е необходимо да се направят поредица от проверки
- Реализация на меню

ПРОГРАМИРАНЕ И ПОВТОРЕНИЕ НА ИЗЧИСЛЕНИЯ

- В реалността съществуват безброй много ситуации при които се налага да се повтарят едни и същи наши действия.
- Например работа на поточна линия в завод, обслужване на клиент на каса в магазин, счетоводство и т.н.
- При програмиране също попадаме в множество такива случаи, например, за да намерим средна стойност се налага да прилагаме операция (многократно) събиране над множество стойности.
- При сортиране и търсене, суми, произведения, намиране на минимални, максимални елементи на множества също.
- Дайте и други примери.
- За да реализираме алгоритми в които се налага да се извършват повтарящи се изчисления в ЕП се използват структури за контрол - т.нар. цикли.

УПРАВЛЯВАЩИ КОНСТРУКЦИИ ЗА ПОВТОРЕНИЕ (ЦИКЛИ)

Операторите за повторение служат за задаване на циклични процеси (цикли, итерации, *loops, iterations*)

- **Особености на циклични процеси:**
 - Те осигуряват неколkokратно повторение на някакви действия, които се наричат **тяло** на цикъла
 - Задължително изискване е циклите да бъдат **крайни**, т.е. броят на повторенията да е крайно число. Това изискване се осигурява от два специални компонента на циклите:
 - условие за край на цикъла – задава кога настъпва края на повторенията
 - управление на броя на повторенията – т.е. цикълът трябва да включва такова действие, което влияе върху стойността на условието за край на цикъла
 - За да се осигури правилността на повторенията, в повечето случаи се налага да се извършат някои предварителни действия, които се наричат **инициализация на цикъла**
- **Основни елементи на итеративните процеси:**
 - инициализация
 - проверка на условието за край на повторенията
 - управление на повторенията
 - повтарящо се действие (тяло)

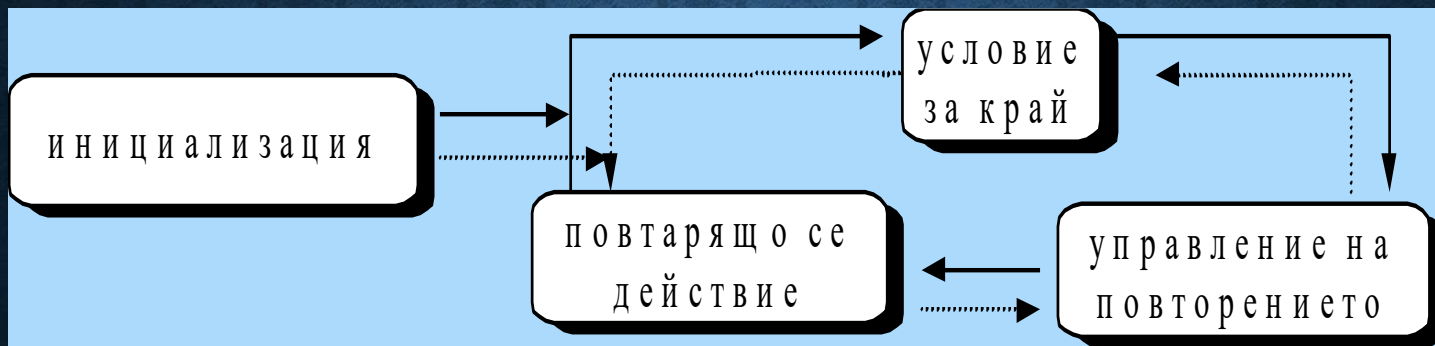
УПРАВЛЯВАЩИ КОНСТРУКЦИИ ЗА ПОВТОРЕНИЕ – ПРИМЕР

Основни елементи на цикъла от алгоритъма на Евклид за намиране на НОД:

Цикъл	Основни елементи
Стъпка 2. Въведете и запомнете числата a и b .	Инициализация
Стъпка 3. Ако $a \neq b$, изпълнете ст. 4, в противен случай – ст. 6.	Проверка на условието за край на повторението
Стъпка 4. Ако $a > b$, то изчислете $a - b$ и го запомнете като a , в противен случай изчислете $b - a$ и го запомнете като b .	Управление на повторенията
Стъпка 5. Изпълнете ст. 3.	Повтарящо се действие

УПРАВЛЯВАЩИ КОНСТРУКЦИИ ЗА ПОВТОРЕНИЕ – ПР. СХЕМА

Основните елементи на цикличните процеси могат да се срещат в различна последователност в конкретни цикли, което на следващата схема е изобразено със стрелки по посока и обратно на часовниковата стрелка:



! Повечето ЕП предоставят различни видове циклични инструкции, които най-често могат да се отнесат към следните два вида:

- оператори за цикъл, при които повторенията на тялото могат да се преброят (обикновено се нарича "for" цикъл)
- оператори за цикъл, при които тялото се изпълнява отново и отново, докато се изпълни някакво условие (обикновено се нарича "while" цикъл)

ОПЕРАТОРИ ЗА ЦИКЪЛ В C++

В C++ има следните основни форми на циклични конструкции:

- Цикли с условие:
 - с предусловие (`while`)
 - със следусловие (`do ... while`)
 - Внимание, тези оператори за цикъл могат да се реализират така, че да се управляват и с т.нар. брояч
- Цикъл, при които повторенията на тялото могат да се преброят преди започването му (`for`), т.е. както се нарича **детерминиран цикъл** (*deterministic loop*), но може да се използва и с по-общо предназначение

ОПЕРАТОРИ ЗА ПОВТОРЕНИЕ В C++: ЦИКЪЛ С ПРЕДУСЛОВИЕ

Същност

Представява итерация, края на повторенията, на която се определя от някакво условие (логическо), а това условие **се изследва преди** всяко повторение на тялото на цикъла

- **Синтаксис:**

```
while (<израз>) <оператор>;
```

където <израз> е логически (Boolean expression) и определя условието за край на цикъла, а <оператор> е кой да е оператор от езика и представлява **тялото** на цикъла. Ако тялото се състои от повече от един оператор, то те се обединяват в един съставен оператор

- **Семантика:**

Изчислява се стойността на <израз> и ако е `true` ($\neq 0$) се изпълнява <оператор>, а ако е `false` ($= 0$) се прекратява изпълнението на цикъла

ЦИКЪЛ С ПРЕДУСЛОВИЕ – ПРИМЕРИ И ОСОБЕНОСТИ

- Примери:

(1)

```
int number = 0;
while (number < 10)
{
    cout << "Loop \n";
    number++;
}
```

(3 безкраен цикъл)

```
int number = 0;
while (number < 5)
    cout << "Hello\n";
```

(2 управление с брояч)

```
s = 0; i = 1;    // инициализация
while (i <= 5)   // проверка на условието за край
{
    // тяло
    s = s + i;
    i = i + 1    // управление на повторенията
}
```

- условието за край се проверява преди изпълнението на тялото на цикъла, което означава, че **тялото на цикъла while може да не се изпълни нито веднъж**
- обикновено се налага за <израз> да се извърши някаква начална инициализация преди влизане в цикъла, за да може да се изчисли неговата стойност
- управлението на повторенията е част от тялото на цикъла. Ето защо в тялото на цикъла `while` трябва да се съдържа поне един оператор, който влияе върху условието за край. В противен случай изпълнението на цикъла никога няма да завърши или ще се получи т.н. безкраен цикъл (зацикляне) – прекрат. Ctrl+C

ПРИМЕРНА ПРОГРАМА

Условие: Да се въвеждат числа до въвеждане на 0 и да се отпечата сумата им:

```
int chislo=1; //иницилизация, за да започне цикъла
int suma=0;
while (chislo!=0)
{
    cin>>chislo;
    suma+=chislo;
}
cout<<suma<<endl;
```

ОПЕРАТОРИ ЗА ПОВТОРЕНИЕ В C++: ЦИКЪЛ СЪС СЛЕДУСЛОВИЕ

Същност

Представява итерация, край на повторенията, на която се определя от някакво условие (логическо), а това условие **се изследва след** всяко повторение на тялото на цикъла

- **Синтаксис:**

```
do <оператор> while (<израз>);
```

, където <израз> е логически(Boolean expression) и определя условието за **край на цикъла**, а <оператор> може да бъде кои да е оператор (може и съставен) от езика и представлява **тялото на цикъла**

- **Семантика:**

1. Изпълнява се <оператор>
2. Изчислява се стойността на <израз>. Ако тя е `true` ($\neq 0$), се повтарят отново стъпки 1 и 2, а ако е `false` ($=0$) се прекратява изпълнението на цикъла

ЦИКЪЛ СЪС СЛЕДУСЛОВИЕ – ПРИМЕРИ И ОСОБЕНОСТИ

- **Примери:**

(1 - с брояч)

```
x = 10;                                     //инициализация
do {                                         //тяло
    y = x * x + 1;
    x--;                                     //управление на повторенията
} while ( x > 0 );                           //проверка на условието за край
```

(2)

```
do {
    cout << "Въведи месец (от 1 до 12) → " ;
    cin >> Month;
} while ((Month < 1) || (Month > 12));
```

- **Особености:**

- **тялото на цикъла се изпълнява поне веднъж**

- управлението на повторенията е част от тялото на цикъла. Ето защо в тялото на цикъла трябва да се съдържа поне един оператор, който влияе върху условието за край. В противен случай изпълнението на цикъла никога няма да завърши или ще се получи т.н. безкраен цикъл (зацикляне)

- операторът се използва за организиране на циклични процеси, при които условието за край може да се провери след изпълнението на повтарящото се действие (например за проверка коректността на входните данни- пр. 2)

ПРИМЕРНА ПРОГРАМА

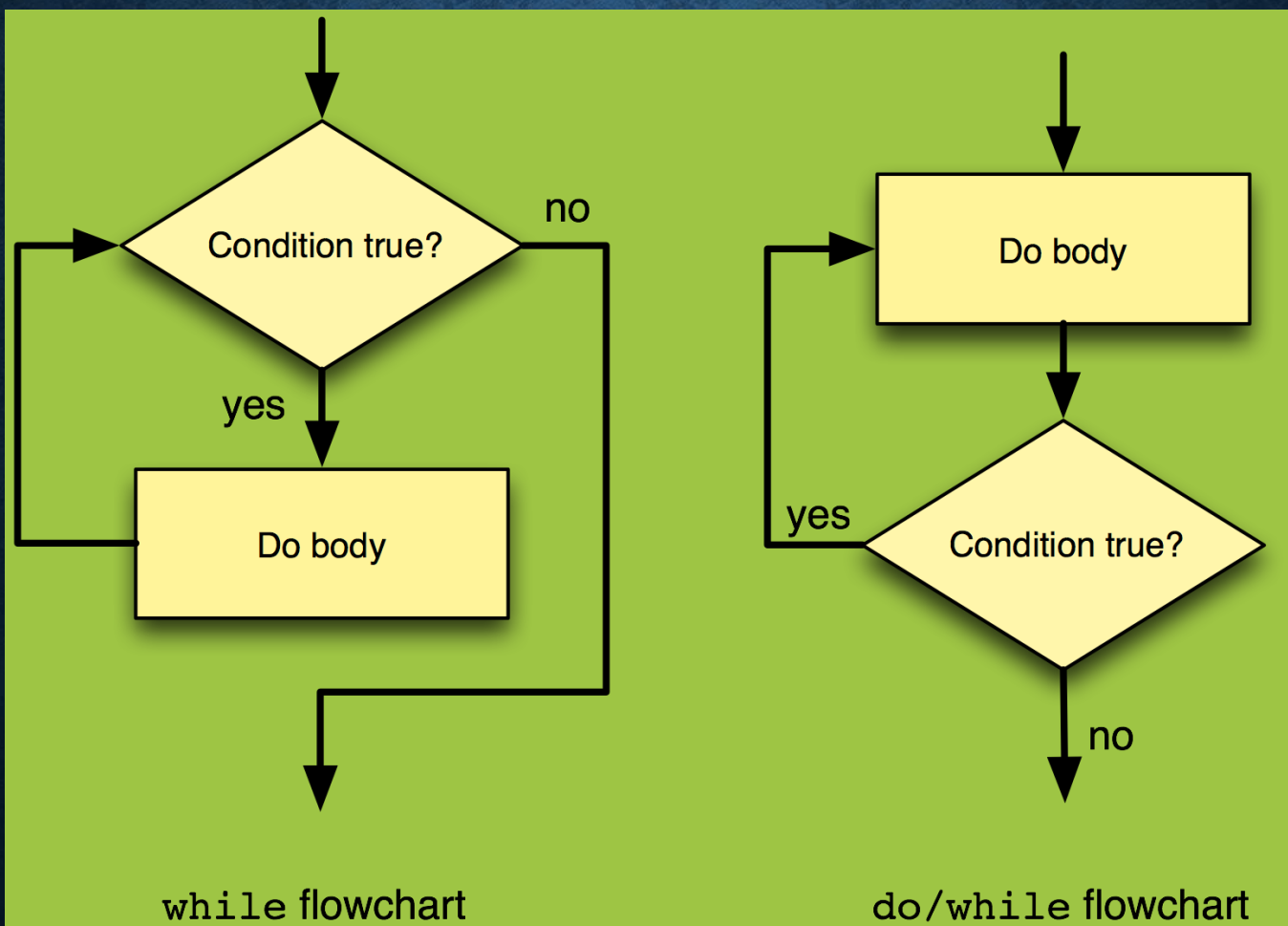
Условие: Да се въвеждат числа до въвеждане на 0 и да се отпечата сумата им:

```
float number, sum = 0.0;

do {
    cout<<"Enter a number: ";
    cin>>number;
    sum += number;
}
while(number != 0.0);

cout<<"Total sum = "<<sum;
```

РАЗЛИЧИЯ В СЕМАНТИКАТА НА WHILE И DO ...WHILE



ОПЕРАТОРИ ЗА ПОВТОРЕНИЕ В C++: ДЕТЕРМИНИРАН ЦИКЪЛ

Същност

Представява итерация, повторенията, на която могат да се преброят преди започването на повторенията

- **Синтаксис:**

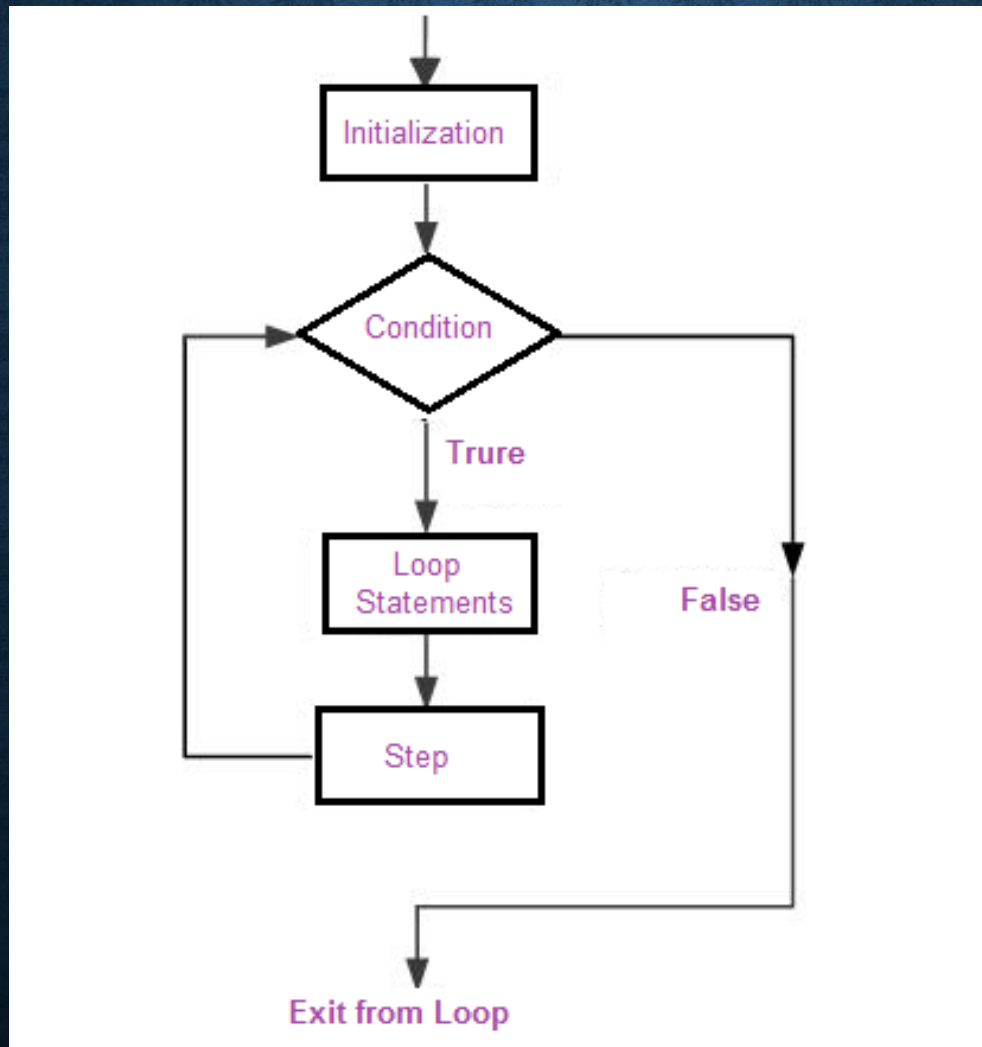
```
for (<инициализиращ_оператор>незад.; <израз1> незад.; <израз2> незад.)  
<оператор>;
```

- <инициализиращ_оператор> обикновено инициализира променлива брояч наричаща се **управляваща променлива на цикъла**. Той може да съдържа последователност от оператори и декларации разделени със запетая
- <израз₁> е булев израз, който обикновено се използва за проверка на условието за край на цикъла
- <израз₂> съдържа код, който обикновено променя текущата стойност на управляващата променлива (управление на повторението)
- <оператор> може да бъде кой да е оператор (може съставен) от езика и представлява тялото на цикъла

- **Семантика:**

1. Изпълнява се <инициализиращ_оператор>
2. Изчислява се <израз₁> и ако е `true`, то: изпълнява се <оператор>, изчислява се <израз₂> и отново се повтаря ст. 2, а ако <израз₁> се оцени като `false`, цикълът се прекратява

ОПЕРАТОРИ ЗА ПОВТОРЕНИЕ В C++: ДЕТЕРМИНИРАН ЦИКЪЛ



ДЕТЕРМИНИРАН ЦИКЪЛ – ПРИМЕРИ И ОСОБЕНОСТИ

• Примери:

(1)

```
for (i = 0; i < 10; i++)  
    sum += i;
```

(2)

```
for (int j = 100; j > 0; j--) {  
    sum += j;  
}
```

(3)

```
for( ; ; )  
    cout<<"Безкраен цикъл";
```

(4)

```
for( int i = 0;  
    i < 100;  
    cout << ++i << endl );
```

• Особености:

- проверката за край на цикъла (т.е. дали управляващата променлива е достигнала крайната си стойност) се извършва преди изпълнението на тялото, което означава, че цикълът може да не се изпълни нито веднъж
- променлива може да бъде декларирана в инициализиращата част на оператора `for`, като нейната област на видимост ще бъде до края на оператора или блока, в който се съдържа този `for`
- `<инициализиращ_оператор>`, `<израз1>` и `<израз2>` не са задължителни (вж. пр. 3)
- Въпреки че трите полета на оператора `for` са предназначени за инициализация, тестване за край и управление на повторението, това не е задължително (вж. пр. 4)

ОПЕРАТОРИ ЗА ПОВТОРЕНИЕ - СЛЕД C++ V.11

В допълнение на вече разгледаните оператори за цикъл след версия C++11 може да се използва и оператор за цикъл базиран на обхват (**range-based for loop**) представлява итерация, върху елементи в даден обхват (колекция, множество)

Знаков модел

- **Синтаксис:**

```
for (<декларация> : <израз>) <оператор>;
```

- < декларация > декларира променлива итератор с тип съответстващ на типа на елементите от колекцията
- <израз> е колекция от елементи

- **Семантика:**

1. Прави се оценка на колекцията (нейния обхват)
2. На всяка итерация на променливата итератор се присвоява поредния елемент от колекцията

```
vector<int> vec; vec.push_back( 10 );  vec.push_back( 20 );  
  
for (int i : vec )  
{  
    cout << i;  
}
```

ОПЕРАТОРИ ЗА БЕЗУСЛОВЕН ПРЕХОД В C++: BREAK И CONTINUE

Операторите за безусловен преход предизвикват незабавно предаване на управлението към конкретен оператор от програмата

Оператор `break`

Оператор `break` обикновено се използва в комбинация с операторите за цикъл или с оператора `switch`, но и с `if` (вж. примера)

- **Семантика:** `break` прекратява изпълнението на цикъла (причинява изход от най-вътрешния цикъл) или на остатъка от блока `switch`. Предава управлението към следващия оператор

- **Пример:**

```
for( ; ; ) // няма условие за край
{
    if (sum >= 1000) break;
    sum += j; j++;}
cout << "Управлението се предава тук.\n";
```

Оператор `continue`

- **Семантика:** `continue` предизвиква следващата итерация (повторение) от цикъла да се стартира веднага, като се игнорира останалата част от текущото повторение.

- **Пример:**

```
for (int j = 100; j > 0; j--) {
    if(j%2 == 0) continue;
    sum += j; }
```

ОПЕРАТОРИ ЗА БЕЗУСЛОВЕН ПРЕХОД В C++: GOTO

Операторът за безусловен преход `goto` може да се използва за да се организира цикъл, както и за прекратяване изпълнението на цикъл, но в по-голямата част от случаите той трябва да се избягва, защото нарушава целостта на структурата на програмата

- **Семантика:** `goto` предизвиква незабавно предаване на управлението към конкретен оператор от програмата, на който е съпоставен етикет. Операторът трябва да бъде от същата функция. Етикетът е идентификатор

- **Пример:**

```
int j = 100, sum = 0;  
LoopStart: sum += j;  
j--;  
if(j > 0) goto LoopStart;
```

СТРУКТУРИ ЗА КОНТРОЛ В ДРУГИ ЕП

- Java – няма съществена разлика нито по отношение на синтаксис нито от семантична гледна точка в сравнение с операторите на C++.
- PHP – има разлика в синтаксиса.

```
while (expr):  
    statement  
    ...  
endwhile;
```

- Python – разлика в синтаксиса.

```
if expression:  
    statement(s)  
else:  
    statement(s)
```